

人工神经网络模型与算法

2025.3 - 2025.5

姓名： 李丰克
专业： 自动化（控制）
年级： 大二
学号： 3230105182

目录

1	Chapter 1 基于numpy手工实现多层感知机模型	3
1.1	问题简介	3
1.2	模型与算法介绍	3
1.2.1	多层感知机模型（MLP）	3
1.2.2	反向传播算法	3
1.2.3	反向传播的计算图简述	4
1.3	算法实现	5
1.4	数据集及算例参数介绍	5
1.4.1	数据集	5
1.4.2	算例参数以及各种优化策略选择	6
1.5	结果汇总	7
1.6	结论	8
1.7	代码附录	8
2	Chapter 2 利用深度学习框架对比微调和从头训练	9
2.1	问题简介	9
2.2	模型与算法介绍	9
2.2.1	ResNeXt	9
2.2.2	ResNeXt的从头训练与预处理调参	9
2.2.3	DenseNet	10
2.2.4	DenseNet的从头训练与预处理调参	10
2.3	数据集	11
2.4	模型训练与结果	11
2.4.1	ResNeXt	11
2.4.2	DenseNet	12
2.5	代码附录	13
3	Chapter 3 基于对抗生成网络GAN实现图象生成	14
3.1	问题简介	14
3.2	模型介绍	14
3.2.1	模型原理	14
3.2.2	代码实现	15
3.3	数据集介绍	15
3.4	训练以及结果	15
3.5	代码附录	17

Chapter 1 基于numpy手工实现多层感知机模型

摘要：本章简述了MLP的原理以及反向传播算法的模型，同时基于python的numpy库实现了该算法，并在FashionMNIST分类数据集上进行了实验，尝试了不同的网络参数、激活函数、优化算法来提高算法的性能以及准确率。

1.1 问题简介

简述反向传播算法的计算图表示方法。

手动搭建ANN并做一做一些其他有趣的分类集合,比如: FashionMNIST, MedMNIST。

尝试不同的网络参数: 全连接网络[784, 28, 10] 或更多, 尝试用不同的激活函数, 尝试用不同的优化算法; 改善程序性能, 给出你的最优方案。

1.2 模型与算法介绍

1.2.1 多层感知机模型 (MLP)

如图1所示, 多层感知机的基本思想为: 前一层的输出作为下一层的输入。在每一层中, 输出的每个神经元等于所有输入的神经元乘以对应的权重再加上对应的偏置项得到的结果再经过激活函数得到的值, 每层的神经元数量可各不相同, 选用的激活函数也各不相同, 因此会出现各种形状的权重矩阵和偏置项, 调整这些权重矩阵以及偏置项参数以提高模型的准确率是我们要做的工作。

假设共有 n 层网络, 用 n 维向量存储每层的神经元个数 $D = [D_0 \ D_1 \ \dots \ D_n]$, 我们令第 K 层的输出为 $X^{(K)}$, 该输出经过激活函数前的值为 $P^{(K)}$, 该层的权重矩阵为 $W^{(K)}$, 偏置向量为 $b^{(K)}$ 。可得:

$$P^{(K)} = W^{(K)}X^{(K-1)} + b^{(K)}, P^{(K)} = f(X^{(K)}), \text{ 其中 } W^{(K)} \text{ 为 } D_k \times D_{k-1} \text{ 的矩阵, } b^{(K)} \text{ 为 } D_k \times 1 \text{ 的向量。}$$

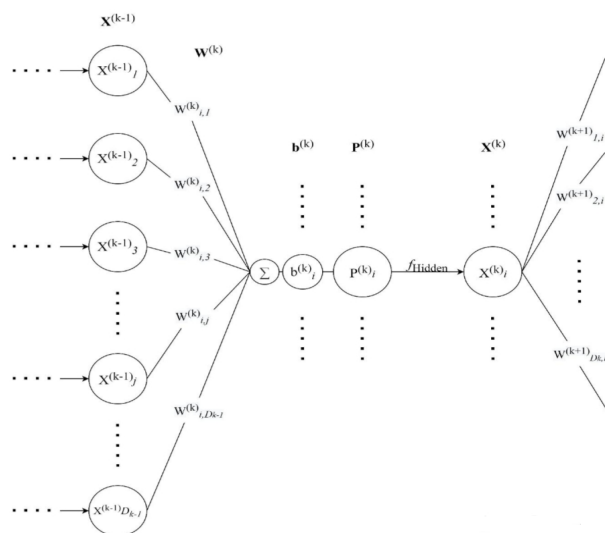


图 1: 多层感知机(第 k 层第 i 个神经元)

中间层统称为隐藏层, 其输出层还会再接入一个损失函数 $\text{Loss}(y)$, 损失函数可以看作模型准确率的指标, 为了使损失函数最小化, 选择用梯度下降法来调整各参数。

1.2.2 反向传播算法

梯度下降法的基本思想为通过计算损失函数相对于每个参数的梯度, 并沿着梯度的反方向更新参数值。MLP本质上可看作输出为 $\text{Loss}(y)$, 输入为 $X^{(0)}$ 的多元函数, 每层每个权重、偏置都是其参数, 所以我们可以求其对每个参数的

偏导再逐步更新参数值。以下是简要推导过程[1, 2]:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^k} = \frac{\partial \mathcal{L}}{\partial \mathbf{P}^k} \cdot \frac{\partial \mathbf{P}^k}{\partial \mathbf{W}^k} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^k} = \frac{\partial \mathcal{L}}{\partial \mathbf{P}^k} \cdot \frac{\partial \mathbf{P}^k}{\partial \mathbf{b}^k}$$

先求 $\partial \mathcal{L} / \partial \mathbf{P}^k$ ，令其等于 δ^k 。

$$\begin{aligned} \delta_i^k &= \frac{\partial \mathcal{L}}{\partial P_i^k} \\ &= \frac{\partial \mathcal{L}}{\partial X_i^k} \cdot \frac{\partial X_i^k}{\partial P_i^k} \\ &= \left(\frac{\partial \mathcal{L}}{\partial P_1^{k+1}} \cdot \frac{\partial P_1^{k+1}}{\partial X_i^k} + \frac{\partial \mathcal{L}}{\partial P_2^{k+1}} \cdot \frac{\partial P_2^{k+1}}{\partial X_i^k} + \dots + \frac{\partial \mathcal{L}}{\partial P_{D_{k+1}}^{k+1}} \cdot \frac{\partial P_{D_{k+1}}^{k+1}}{\partial X_i^k} \right) (f'(P_i^k)) \\ &= \left(\sum_{j=1}^{D_{k+1}} \frac{\partial \mathcal{L}}{\partial P_j^{k+1}} \cdot \frac{\partial P_j^{k+1}}{\partial X_i^k} \right) (f'(P_i^k)) \\ &= \left(\sum_{j=1}^{D_{k+1}} \frac{\partial \mathcal{L}}{\partial P_j^{k+1}} \cdot \frac{\partial (\sum_{m=1}^{D_k} W_{j,m}^{k+1} \cdot X_m^k + b_j^{k+1})}{\partial X_i^k} \right) (f'(P_i^k)) \\ &= \left(\sum_{j=1}^{D_{k+1}} \frac{\partial \mathcal{L}}{\partial P_j^{k+1}} \cdot \frac{\partial (W_{j,1}^{k+1} \cdot X_1^k + W_{j,2}^{k+1} \cdot X_2^k + \dots + W_{j,i}^{k+1} \cdot X_i^k + \dots + W_{j,D_k}^{k+1} \cdot X_{D_k}^k + b_j^{k+1})}{\partial X_i^k} \right) (f'(P_i^k)) \\ &= \left(\sum_{j=1}^{D_{k+1}} \delta_j^{k+1} \cdot W_{j,i}^{k+1} \right) (f'(P_i^k)) \end{aligned}$$

可得 δ_i^k 是行向量 δ^{k+1} 和列向量 $\mathbf{W}_{:,i}^{k+1}$ 的点乘再乘激活函数对应的偏导，因此有 $\delta^k = (\delta^{k+1} \cdot \mathbf{W}^{k+1}) \odot f'(\mathbf{P}^k)$ 。发现有自后向前的迭代关系。然后再代回偏导公式:

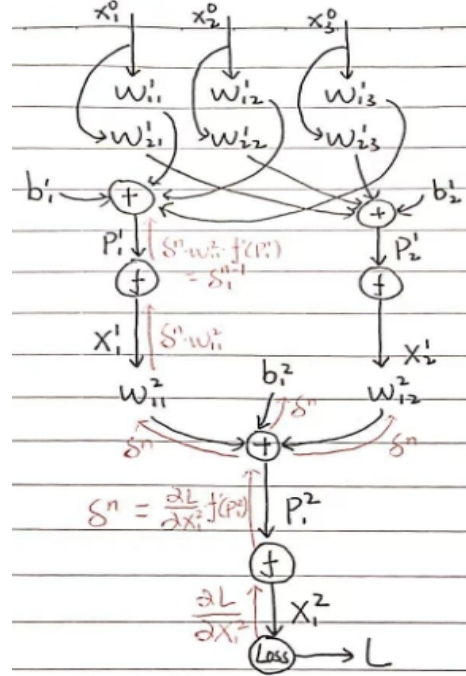
$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_{i,j}^k} &= \frac{\partial \mathcal{L}}{\partial P_i^k} \cdot \frac{\partial P_i^k}{\partial W_{i,j}^k} \\ &= \delta_i^k \cdot \frac{\partial (W_{i,1}^k \cdot X_1^{k-1} + W_{i,2}^k \cdot X_2^{k-1} + \dots + W_{i,j}^k \cdot X_j^{k-1} + \dots + W_{i,D_{k-1}}^k \cdot X_{D_{k-1}}^{k-1} + b_i^k)}{\partial W_{i,j}^k} \\ &= \delta_i^k \cdot X_j^{k-1} \\ \frac{\partial \mathcal{L}}{\partial b_i^k} &= \frac{\partial \mathcal{L}}{\partial P_i^k} \cdot \frac{\partial P_i^k}{\partial b_i^k} \\ &= \delta_i^k \cdot \frac{\partial (W_{i,1}^k \cdot X_1^{k-1} + W_{i,2}^k \cdot X_2^{k-1} + \dots + W_{i,j}^k \cdot X_j^{k-1} + \dots + W_{i,D_{k-1}}^k \cdot X_{D_{k-1}}^{k-1} + b_i^k)}{\partial b_i^k} \\ &= \delta_i^k \end{aligned}$$

最终得到

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{W}^k} &= (\delta^k)^T \cdot (\mathbf{X}^{k-1})^T \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}^k} &= (\delta^k)^T \end{aligned}$$

故要先解得损失函数对于输出层输出的偏导，然后再以此递推。

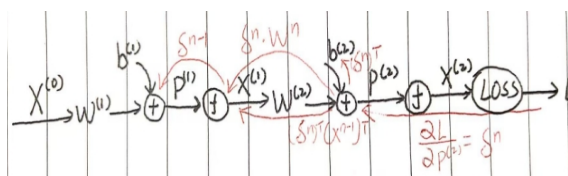
1.2.3 反向传播的计算图简述



如图是手绘的一个三层神经网络的计算图，其神经元个数分别为3，2，1。该计算图中共有三种节点：加法器，乘法器和函数，前向传播时对其进行对应的运算，根据链式法则，反向传播时，每经过一个节点乘上对应的偏导运算。经过加法器时偏导为1；经过乘法器时偏导为放大增益；经过函数时，偏导为函数对运算值的偏导数。

从图中可以看出 δ^n 可直接求，从 P_1^2 经过加法器的过程中，可以直接得到 b_1^2 ，这符合前面对 $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^k} = (\delta^k)^T$ 的推导。在经过乘法器时，将对应的X看作增益，即可得到对W的偏导，符合 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^k} = (\delta^k)^T \cdot (\mathbf{X}^{k-1})^T$ 。将W看为增益，再经过函数，便可得到下一个 δ^{n-1} ，符合 $\delta^k = (\delta^{k+1} \cdot \mathbf{W}^{k+1}) \odot f'(\mathbf{P}^k)$ 的推导。

1 Chapter 1 基于numpy手工实现多层感知机模型



如图用矩阵表示计算图的各环节，其中各层的偏导数的迭代关系则一目了然，再按照矩阵的运算规律适当变形，便可得出相同的推导规律。

```
#定义反向传播函数
def backpropagation(weights, thetas, activation, activation_deriv, x, y):
    #error是损失函数对最后一次输出的直接偏导数，这个与损失函数的选择有关，这里选择用
    #均方误差函数，其输出求偏导结果为y_pred-y_true，即输出值-真实值
    error=0
    y_true=propagation(weights, thetas, activation, x, 1)
    for i in range(len(y)):
        error+=y_true[i][0]-y[i]
    n=len(weights)
    z=[]#每一层的未经激活函数的输出，即p
    K=[]#每一层经过激活函数的输出，从输入层开始
    dweights=[]#损失函数L对每个权重的偏导W的矩阵
    dthetas=[]#损失函数L对每个偏置b的偏导的矩阵
    delta=[]#损失函数L对每一层输出p的偏导
    for i in range(n_w):#前向传播得到每层输出
        if i==0:
            z.append(np.dot(weights[i], x) + thetas[i])
            K.append(activation(z))
            delta.append(x)
        else:
            z.append(np.dot(weights[i], activation(z[i-1])) + thetas[i])
            K.append(activation(z[i-1]))
            delta.append(activation(z[i-1]))
    for i in range(n_w-1, -1, -1):#得到全部delta
        if i==n_w-1:
            delta[i] = np.multiply(error, activation_deriv(z[i]))#得到delta^n
        else:
            delta[i] = np.multiply(np.dot(weights[i+1].T, delta[i+1]), activation_deriv(z[i]))
            #依次迭代得到每个delta
    for i in range(n_w):#得到全部偏导
        dweights.append(np.dot(delta[i], K[i].T))
        dthetas.append(delta[i])
    return dweights, dthetas
```

1.3 算法实现

为了实现MLP，我们需要事先准备：神经网络层数及各层神经元个数，前向传播函数，反向传播函数，各种激活函数及其导数，各种优化算法，损失函数（这里默认为均方误差）。

定义了 $\tanh(x)$ ， $\text{sigmoid}(x)$ ， $\text{RELU}(x)$ ， $\text{RELU3}(x)$ 四种激活函数及其导数。权重矩阵与偏置矩阵的初始化如下：

```
#权重与偏置初始化
def WandBInit(layers):
    weights=[]
    thetas=[]
    for i in range(len(layers)-1):
        weights.append(2*(np.random.rand(layers[i+1], layers[i]))-1)
        thetas.append(2*(np.random.rand(layers[i+1]))-1)
        thetas[i]=np.asmatrix(thetas[i]).T
    return weights, thetas
```

反向传播函数如下：

梯度下降优化函数如下：

```
#定义梯度下降优化算法
def GD(weights, thetas, layers, X, Y, k, learning_rate, activation, activation_deriv):
    perf=0#perf是总损失
    ddweights=[]#更新后的权重
    ddthetas=[]#更新后的偏置
    for i in range(len(layers)-1):#形状初始化
        ddweights.append(np.zeros((layers[i+1], layers[i])))
        ddthetas.append(np.zeros(layers[i+1]))
        ddthetas[i] = np.asmatrix(ddthetas[i]).T
    for j in range(int(X.shape[1])):
        input = np.asmatrix(X[:,j]).T
        output = propagation(weights, thetas, activation, input, 1)
        y = np.asmatrix(Y[:,j]).T
        error = y - output
        perf += 1.0/2 * np.sum(np.multiply(error, error))#均方误差损失函数
        dweights, dthetas = backpropagation(weights, thetas, activation, activation_deriv, x, y)
        for i in range(len(weights)):#更新参数
            ddweights[i] += k * 1.0/len(X)*learning_rate * dweights[i]
            ddthetas[i] += k * 1.0/len(X)*learning_rate * dthetas[i]
        for i in range(len(weights)):#赋值
            weights[i] += ddweights[i]
            thetas[i] += ddthetas[i]
    return perf, weights, thetas
```

前向传播以及其他优化算法均见代码附录。

1.4 数据集及算例参数介绍

1.4.1 数据集

选用的数据集为FashionMNIST，是一个分类数据集，其包含60000个训练样本和10000个测试样本，每个样本都是 28×28 的灰度图，其标签从0到9分别对应了10种时尚产品。

从本地加载数据，由于数据包为gz压缩，调用gzip库解压缩并将训练图像和测试图像reshape为 28×28 的灰度图矩阵，可以抽取前若干张用matplotlib库输出图像并看是否与标签对应，保证数据正确引用。

引用后得到的train-images是一个 $60000 \times 28 \times 28$ 的三维数据，将每个 28×28 数据reshape为 784×1 的列向量依次作为输入，train-labels作为标签reshape为 60000×1 的向量，测试集同理。

由于标签涉及0-9，激活函数值域不够，最后输出时需要相应的放大倍数k，比如对sigmoid函数来说k取10。

1.4.2 算例参数以及各种优化策略选择

共有三个变量：优化算法，网络参数，激活函数。受限于电脑性能，每次迭代时间太长，因此在决定使用哪种策略更好的时候并不需要把迭代次数设置的太多以使得loss最小，准确率最高，可以通过控制变量的方法来确定，暂且假设三个因素是相互独立的。同时权重与偏置只初始化一次，确保所有实验是在同一个基础上开始的。

选择优化算法：对于GD(批量梯度下降)算法来说，本数据集数据量太大了，无论是从CPU占用，还是迭代时间上来说，本电脑都无法承受，因此选择取数据集的前1%来进行训练；对于SGD(随机梯度下降)，由于其原理是随机抽取小批量样本进行随机训练，虽然训练过程不稳定，多少效率高，可以对全部数据进行训练；对于ADAM，其过程中也抽取

了一部分样本进行随机训练，速度略快于GD。因此定为：

网络参数	[784,256,128,1]
激活函数	sigmoid
放大倍数	k=10
学习率	0.1
更新缩放因子	m=1
迭代次数	1000

其中对于GD选取前1%的数据，SGD与ADAM选择全部数据。

选择网络参数：由于SGD与ADAM具有随机性，选用GD作为优化算法，网络参数分为两个极端：一种是层数少，相邻层间跨度大；另一种是层数多，相邻层间平滑过渡。因此可以选择三种网络层数[784,256,1],[784,256,128,1],[784,392,196,49,1]。其他参数如下：

优化算法	GD
激活函数	sigmoid
放大倍数	k=10
学习率	0.1
更新缩放因子	m=1
迭代次数	1000

选择激活函数：激活函数共有四个，分别为tanh，sigmoid，RELU，RELU3。对于tanh与sigmoid函数放大倍数要选k=10，其余两个k=1即可。其他参数如下：

优化算法	GD
网络参数	[784,256,128,1]
学习率	0.1
更新缩放因子	m=1
迭代次数	1000

由此可以组合出性能较佳的参数，但是不一定是最佳的，因为算法中存在

的随机性，或者各条件直接并不是绝对的独立关系，或者与数据集本身特点有关等等。

1.5 结果汇总

事实证明，仅仅一千次迭代结果并不能得到较高的准确率，但是足以反映各参数之间的区别。

以优化算法为变量：

```
Epoch 994: Loss = 210.0200038145937
Epoch 995: Loss = 209.81944492602048
Epoch 996: Loss = 209.61920815033838
Epoch 997: Loss = 209.41929290312945
Epoch 998: Loss = 209.21969859901085
Epoch 999: Loss = 209.02042465163439
```

```
Epoch 950: Loss = 919.8444236971885
Epoch 960: Loss = 787.0678279017072
Epoch 970: Loss = 803.1193932931988
Epoch 980: Loss = 884.1280940442069
Epoch 990: Loss = 910.0213626553469
```

```
Epoch 996: Loss = 8657.5
Epoch 997: Loss = 8289.5
Epoch 998: Loss = 8239.0
Epoch 999: Loss = 8086.5
```

自上而下依次为GD，SGD，ADAM三种优化算法的迭代情况，由于对损失函数的取值不同。ADAM取数据集的百分之十而SGD取百分之一，故刚好差1个数量级。SGD与ADAM的收敛过程为振荡收敛，而GD为单调收敛。收敛速度SGD是最快的，而GD是最慢的(如果以全部数据来训练)。

三者测试集上的准确率依次为0.2447, 0.2485, 0.2515，GD的训练集太小，泛化能力不够，如果让其在整个数据集上训练，训练时间太长，耗费太大，但是一定能够收敛到一个较合适的值。

SAG迭代速度快，适合处理大数据，且准确率也尚可。ADAM的准确率最高，说明其收敛性最好，得益于其自动调整学习率的算法原理。

以激活函数为变量：

```
Epoch 994: Loss = 210.0200038145937
Epoch 995: Loss = 209.81944492602048
Epoch 996: Loss = 209.61920815033838
Epoch 997: Loss = 209.41929290312945
Epoch 998: Loss = 209.21969859901085
Epoch 999: Loss = 209.02042465163439
```

```
Epoch 995: Loss = 7957.346738689841
Epoch 996: Loss = 7523.692390601167
Epoch 997: Loss = 11524.001073726467
Epoch 998: Loss = 11340.591088827683
Epoch 999: Loss = 10704.54210360599
```

```
Epoch 995: Loss = 6009.0
Epoch 996: Loss = 6009.0
Epoch 997: Loss = 6009.0
Epoch 998: Loss = 6009.0
Epoch 999: Loss = 6009.0
Epoch 995: Loss = 6009.0
Epoch 996: Loss = 6009.0
Epoch 997: Loss = 6009.0
Epoch 998: Loss = 6009.0
Epoch 999: Loss = 6009.0
```

依次为sigmoid,tanh,RELU,RELU3为激活函数对应的迭代过程。奇怪的是，RELU与RELU3作为激活函数时，总会陷入死循环，使神经元失效，可能与数据集自身有关，也可能在算法某些过程上出现了问题，目前仍未排查。

对tanh函数，其迭代过程始终在振荡，损失在7000 12000的区间内振荡，始终不能再下降，推测应该是学习率设置的太高的问题，将学习率设置为0.01，再跑一次。

```
Epoch 995: Loss = 2970.667548326755
Epoch 996: Loss = 2592.9146275539783
Epoch 997: Loss = 2969.613239908594
Epoch 998: Loss = 2592.140719720677
Epoch 999: Loss = 2968.5524827996396
```

发现整体的损失减小了，但是仍然存在振荡不收敛的现象，说明学习率仍然高，只要再适当减小学习率，理论上可以收敛。这也说明了tanh函数受学习率影响较大，适用于较小学习率，这也使得其收敛速度变慢。目前来说，还是sigmoid性能最好。

以网络参数为变量：

```
Epoch 995: Loss = 828.970839980209
Epoch 996: Loss = 750.9533274225571
Epoch 997: Loss = 828.3051932309572
Epoch 998: Loss = 750.281835245485
Epoch 999: Loss = 827.6397561278371
```

```
Epoch 994: Loss = 210.0200038145937
Epoch 995: Loss = 209.81944492602048
Epoch 996: Loss = 209.61920815033838
Epoch 997: Loss = 209.41929290312945
Epoch 998: Loss = 209.21969859901085
Epoch 999: Loss = 209.02042465163439
```

```
Epoch 995: Loss = 273.0550110958664
Epoch 996: Loss = 272.7778542055008
Epoch 997: Loss = 272.5010582501921
Epoch 998: Loss = 272.22462291103255
Epoch 999: Loss = 271.94854786671107
```

自上而下依次为[784,256,1]，[784,256, 128,1]，[784,392,196,49,1]对应的迭代过程。由此可以发现，神经网络层数越多，层间过渡越平稳，参数收敛越快，性能越好，但是相应的每次迭代所花费的时间也越多

```
749.6116537645371
0.1489
```

```
208.82147047368906
0.2447
```

```
749.6116537645371
0.1489
```

```
208.82147047368906
0.2447
```

```
271.6728327935459
0.3215
```

以上为三种参数在测试集上的损失以及准确率，迭代次数有限情况下，准确率与收敛速度挂钩，与前面的规律也相吻合。如果不考虑时间花费的话，一定是[784,392,196,49,1]性能最优。如果要综合考虑数据集大小、时间花费的话，还是选择折中的[784,256,128,1]较好。

1.6 结论

综上，在三个变量相互独立的假设下，选择[784,256,128,1]，sigmoid激活函数，ADAM优化算法的优化策略，使模型无论从收敛速度，准确率，时间花费上都最佳。

其他的参数为：放大倍数k=10，缩放因子m=1，学习率0.1，迭代次数10000（尽量大）。

1.7 代码附录

代码的zjugit网址：<https://git.zju.edu.cn/3230105182/mlp>

参考文献

- [1] Nielsen M. Neural Networks and Deep Learning[M]. San Francisco: Determination Press, 2015.
- [2] Johnson J. CS231n: Convolutional Neural Networks for Visual Recognition[EB/OL]. Stanford: Stanford University, 2019. [2025-03-16]. <https://cs231n.github.io/>.

Chapter 2 利用深度学习框架对比微调和从头训练

2.1 问题简介

以ResNeXt和DenseNet为例，自行选择数据集，按照从头训练和模型预处理后微调两种方式去训练模型，要求训练时间不低于两小时。

2.2 模型与算法介绍

2.2.1 ResNeXt

ResNeXt是在残差网络模型ResNet的基础上发展起来的，引入了分组卷积的概念，将输入层的不同特征图先进行分组，然后分别采用不同的卷积核进行卷积，这样虽然卷积核的总数量没变，但是总参数量减少了，减小了卷积的运算量。

stage	output	ResNet-50	ResNeXt-50 (32×4d)
conv1	112×112	7×7, 64, stride 2	7×7, 64, stride 2
conv2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128, C=32 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256, C=32 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512, C=32 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 1024 \\ 3 \times 3, 1024, C=32 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax

上图为ResNet与ResNeXt的各层结构。可以发现两者的输入层、最大池化层、输出层均相同。中间的主体卷积层不同，不同点在于分组卷积。

输入层：7×7卷积，步长为2，输入为三通道RGB图像，输出为64通道。

最大池化层：3×3卷积，步长为2。

中间四层卷积：layer1：有3个ResNeXt模块，每个模块的输出通道数为256，每个分组卷积通道数64，基数为32。layer2：4个ResNeXt模块，每个模块输出通道数512，每个分组卷积通道数128，基数32。layer3：6个ResNeXt模块，每个模块输出通道数1024，每个分组卷积通道数256，基数32。layer4：3个ResNeXt模块，每个模块输出通道数2048，每个分组卷积通道数512，基数32。

输出层：全局平均池化层，全连接层，softmax层。

基于分组卷积大大减小的计算量，而且每层模块有相同的拓扑结构，提高了模型的通用性。

本次使用的模型为resnext50_32x4d。

2.2.2 ResNeXt的从头训练与预处理调参

首先要定义主体的每个resnext模块，因为使用的resnext50模型较为复杂，选用Bottleneck。

```
class Bottleneck(nn.Module):
    expansion=4
    def __init__(self, in_channel, out_channel, stride=1, downsample=None, groups=1, width_per_group=64):
        super(Bottleneck, self).__init__()
        width=int(out_channel*(width_per_group/64.))*groups
        #第一层卷积
        self.conv1 = nn.Conv2d(in_channels=in_channel, out_channels=width, kernel_size=1, stride=1, bias=False)
        self.bn1 = nn.BatchNorm2d(width)
        #第二层中间层
        self.conv2 = nn.Conv2d(in_channels=width, out_channels=width, groups=groups, kernel_size=3, stride=stride, bias=False, padding=1)
        self.bn2 = nn.BatchNorm2d(width)
        #第三层卷积
        self.conv3 = nn.Conv2d(in_channels=width, out_channels=out_channel*self.expansion, kernel_size=1, stride=1, bias=False)
        self.bn3 = nn.BatchNorm2d(out_channel * self.expansion)
        self.relu=nn.ReLU(inplace=True)
        self.downsample=downsample
```

然后定义resnext整体结构，包括输入层、池化层、主题层和输出层。

2 Chapter 2 利用深度学习框架对比微调和从头训练

```
class ResNetXt(nn.Module):
    def __init__(self, block, block_num, num_classes=1000, include_top=True, groups=1, width_per_group=64):
        super(ResNetXt, self).__init__()
        self.include_top=include_top
        self.in_channel=64
        self.groups=groups
        self.width_per_group=width_per_group

        self.conv1 = nn.Conv2d(3, self.in_channel, kernel_size=7, stride=2, padding=3, bias=False)
        self.bn1=nn.BatchNorm2d(self.in_channel)
        self.relu=nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        self.layer1=self._make_layer(block, 64, block_num[0])
        self.layer2=self._make_layer(block, 128, block_num[1], stride=2)
        self.layer3=self._make_layer(block, 256, block_num[2], stride=2)
        self.layer4=self._make_layer(block, 512, block_num[3], stride=2)
```

对于resnext50_32×4d，分组卷积的分组数为32，每组的通道数为4。

从此基础上开始训练，即对resnext50从头训练。

在torchvision库中的models里，有预训练好的resnext50_32×4d模型，但是其默认的输出特征数为1000，即分类得到的类别数，可以根据数据集来调整输出特征数。

```
from torchvision import models
resnext50 = models.resnext50_32x4d(pretrained=True)
num_fters = resnext50.fc.in_features
for param in resnext50.parameters():
    param.requires_grad = False #False: 冻结模型的参数,

#保持in_features不变, 修改out_features=10
resnext50.fc = nn.Sequential(nn.Linear(num_fters, 2),
                             nn.LogSoftmax(dim=1))
```

这样得到的模型即为预训练好的模型。

2.2.3 DenseNet

DenseNet(稠密神经网络),其与ResNet相比，将所有层连接，每个层都会接受其前面所有层作为其额外的输入，采用密集连接的方式来加快训练速度。

Layers	Output Size	DenseNet-121	DenseNet-169	DenseNet-201	DenseNet-264
Convolution	112 × 112	7 × 7 conv, stride 2			
Pooling	56 × 56	3 × 3 max pool, stride 2			
Dense Block (1)	56 × 56	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$
Transition Layer (1)	56 × 56	1 × 1 conv			
	28 × 28	2 × 2 average pool, stride 2			
Dense Block (2)	28 × 28	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$
Transition Layer (2)	28 × 28	1 × 1 conv			
	14 × 14	2 × 2 average pool, stride 2			
Dense Block (3)	14 × 14	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 24$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 64$
Transition Layer (3)	14 × 14	1 × 1 conv			
	7 × 7	2 × 2 average pool, stride 2			
Dense Block (4)	7 × 7	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 16$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$
Classification Layer	1 × 1	7 × 7 global average pool			
		1000D fully-connected, softmax			

如图为DenseNet的结构，首先经过了7×7的卷积层和3×3的最大池化层，然后进入了网络的主体结构，主体结构由DenseBlock和Transition层构成，然后通过全局平均池化层、全连接层、softmax层输出。

DenseBlock中，各个层的特征图大小一致，可以在channel维度上连接。DenseBlock中的非线性组合函数采用的是BN+ReLU+3x3 Conv的结构。由于后面层的输入会非常大，DenseBlock内部可以采用bottleneck层来减少计算量，即在原有的结构中增加1x1 Conv，提升计算效率。

Transition层，主要作用是连接两个相邻的DenseBlock，并且降低特征图大小。Transition层包括一个1x1的卷积层和2x2的平均池化层，结构为BN+ReLU+1x1Conv+2x2AvgPooling。此外其也可以起到压缩模型的作用。

本次用到的模型为densenet121模型。

2.2.4 DenseNet的从头训练与预处理调参

首先定义denselayer，即为后续结果的单元模块。

```
class _DenseLayer(nn.Sequential):
    def __init__(self, num_input_features, growth_rate, bn_size, drop_rate):
        super(_DenseLayer, self).__init__()
        self.add_module("norm1", nn.BatchNorm2d(num_input_features))
        self.add_module("relu1", nn.ReLU(inplace=True))
        self.add_module("conv1", nn.Conv2d(num_input_features, bn_size*growth_rate,
                                             kernel_size=1, stride=1, bias=False))
        self.add_module("norm2", nn.BatchNorm2d(bn_size*growth_rate))
        self.add_module("relu2", nn.ReLU(inplace=True))
        self.add_module("conv2", nn.Conv2d(bn_size*growth_rate, growth_rate,
                                             kernel_size=3, stride=1, padding=1, bias=False))
        self.drop_rate = drop_rate

    def forward(self, x):
        new_features = super(_DenseLayer, self).forward(x)
        if self.drop_rate > 0:
            new_features = F.dropout(new_features, p=self.drop_rate, training=self.training)
        return torch.cat([x, new_features], 1)
```

然后定义DenseBlock和Transition层。

```
class _DenseBlock(nn.Sequential):
    def __init__(self, num_layers, num_input_features, bn_size, growth_rate, drop_rate):
        super(_DenseBlock, self).__init__()
        for i in range(num_layers):
            layer = _DenseLayer(num_input_features+i*growth_rate, growth_rate, bn_size,
                                drop_rate)
            self.add_module("denseLayer%d" % (i+1), layer)

class _Transition(nn.Sequential):
    def __init__(self, num_input_feature, num_output_features):
        super(_Transition, self).__init__()
        self.add_module("norm", nn.BatchNorm2d(num_input_feature))
        self.add_module("relu", nn.ReLU(inplace=True))
        self.add_module("conv", nn.Conv2d(num_input_feature, num_output_features,
                                             kernel_size=1, stride=1, bias=False))
        self.add_module("pool", nn.AvgPool2d(2, stride=2))
```

最后定义整体结构，加上输入层的卷积层和最大池化层与输出层。

```
from collections import OrderedDict
class DenseNet(nn.Module):
    def __init__(self, growth_rate=32, block_config=(6, 12, 24, 16), num_init_features=64,
                 bn_size=4, compression_rate=0.5, drop_rate=0, num_classes=1000):
        super(DenseNet, self).__init__()
        # first Conv2d
        self.features = nn.Sequential(OrderedDict([
            ("conv0", nn.Conv2d(3, num_init_features, kernel_size=7, stride=2, padding=3, bias=False)),
            ("norm0", nn.BatchNorm2d(num_init_features)),
            ("relu0", nn.ReLU(inplace=True)),
            ("pool0", nn.MaxPool2d(3, stride=2, padding=1))
        ]))

        # DenseBlock
        num_features = num_init_features
        for i, num_layers in enumerate(block_config):
            block = _DenseBlock(num_layers, num_features, bn_size, growth_rate, drop_rate)
            self.features.add_module("denseBlock%d" % (i + 1), block)
            num_features += num_layers*growth_rate
            if i != len(block_config) - 1:
                transition = _Transition(num_features, int(num_features*compression_rate))
                self.features.add_module("transition%d" % (i + 1), transition)
                num_features = int(num_features * compression_rate)

        # final bn+ReLU
        self.features.add_module("norm5", nn.BatchNorm2d(num_features))
        self.features.add_module("relu5", nn.ReLU(inplace=True))

        # classification layer
        self.classifier = nn.Linear(num_features, num_classes)

        # params initialization
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight)
```

从此基础上开始训练，即对resnext50从头训练。

而torchvision库中的models里，也有预训练好的DenseNet121模型，修改其输出特征数即可得到预处理好的模型。

```
from torchvision import models
densenet121 = models.densenet121(pretrained=True)
num_fts = densenet121.classifier.in_features
for param in densenet121.parameters():
    param.requires_grad = False #False, 冻结模型的

#保持in_features不变, 修改out_features=2
densenet121.classifier = nn.Linear(num_fts, 2)
```

2.3 数据集

本次选用的数据集为2013年kaggle里的经典竞赛题cats vs dogs。其中共包含25000张图片，猫狗图片比例1:1，分类任务为分辨出猫狗。

将图片文件保存在本地，用shutil库按照4:1的比例分出训练集和测试集分别存在两个文件夹，用torchvision库中的datasets读取数据，得到训练集与测试集的dataset类型变量，其中cat的标签为0，dog的标签为1。再用torch库中的dataloader方法加载数据集。其中batch_size(每次加载的样本数量)设置为64，num_worker(数据加载的子进程数量)根据cpu情况自行调整。

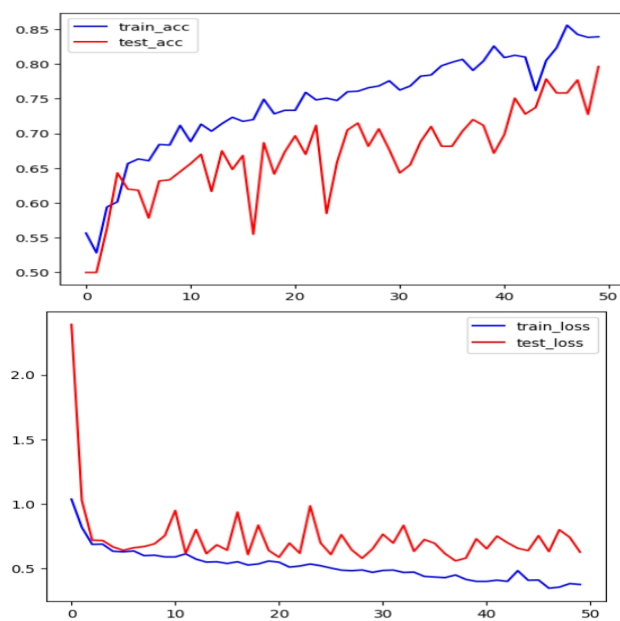
2.4 模型训练与结果

2.4.1 ResNeXt

对resnext50_32×4d分别进行从头训练和预处理训练，学习率为0.001，优化算法为Adam，损失函数为交叉熵损失函数。

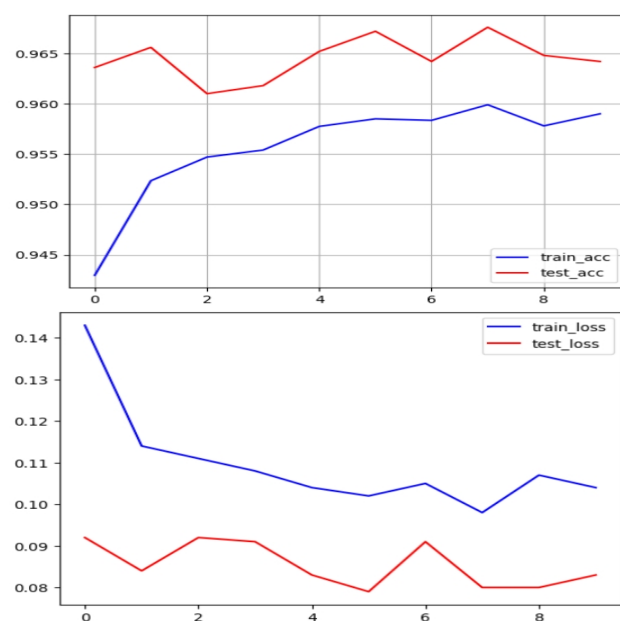
对从头训练的模型，训练50轮，用时大约4h，结果如下。

```
Epoch:45, Train_acc:80.5%, Train_loss:0.407, Test_acc:77.8%, Test_loss:0.637, Lr:1.00E-03
train over
Epoch:46, Train_acc:82.3%, Train_loss:0.409, Test_acc:75.8%, Test_loss:0.753, Lr:1.00E-03
train over
Epoch:47, Train_acc:85.6%, Train_loss:0.345, Test_acc:75.8%, Test_loss:0.630, Lr:1.00E-03
train over
Epoch:48, Train_acc:84.2%, Train_loss:0.354, Test_acc:77.7%, Test_loss:0.798, Lr:1.00E-03
train over
Epoch:49, Train_acc:83.8%, Train_loss:0.381, Test_acc:72.8%, Test_loss:0.740, Lr:1.00E-03
train over
Epoch:50, Train_acc:83.9%, Train_loss:0.375, Test_acc:79.6%, Test_loss:0.625, Lr:1.00E-03
Done
```



对预处理的模型，训练10轮，用时大约0.5h，结果如下。

```
train over
Epoch: 1, Train_acc:94.3%, Train_loss:0.143, Test_acc:96.4%, Test_loss:0.092, Lr:1.00E-03
train over
Epoch: 2, Train_acc:95.2%, Train_loss:0.114, Test_acc:96.6%, Test_loss:0.084, Lr:1.00E-03
train over
Epoch: 3, Train_acc:95.5%, Train_loss:0.111, Test_acc:96.1%, Test_loss:0.092, Lr:1.00E-03
train over
Epoch: 4, Train_acc:95.5%, Train_loss:0.108, Test_acc:96.2%, Test_loss:0.091, Lr:1.00E-03
train over
Epoch: 5, Train_acc:95.8%, Train_loss:0.104, Test_acc:96.5%, Test_loss:0.083, Lr:1.00E-03
train over
Epoch: 6, Train_acc:95.9%, Train_loss:0.102, Test_acc:96.7%, Test_loss:0.079, Lr:1.00E-03
train over
Epoch: 7, Train_acc:95.8%, Train_loss:0.105, Test_acc:96.4%, Test_loss:0.091, Lr:1.00E-03
train over
Epoch: 8, Train_acc:96.0%, Train_loss:0.098, Test_acc:96.8%, Test_loss:0.080, Lr:1.00E-03
train over
Epoch: 9, Train_acc:95.8%, Train_loss:0.107, Test_acc:96.5%, Test_loss:0.080, Lr:1.00E-03
train over
Epoch:10, Train_acc:95.9%, Train_loss:0.104, Test_acc:96.4%, Test_loss:0.083, Lr:1.00E-03
Done
```



可以看出从头训练时，五十轮后在测试集上有大约80%的准确率。而预处理的模型初始便有96%的准确率，所以这里只训练了十轮，受限于电脑性能，如果将训练轮数提高到100轮或以上，可能能够达到90%以上的准确率。

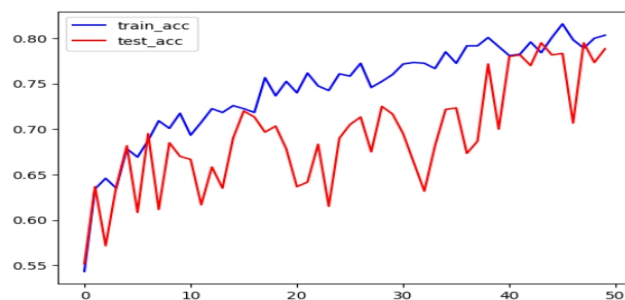
虽然没有在图中表示出来，但是在训练过程中可以看出，使用预处理的模型每轮的训练时间显著少于从头训练的。这可能与结构也有关系。

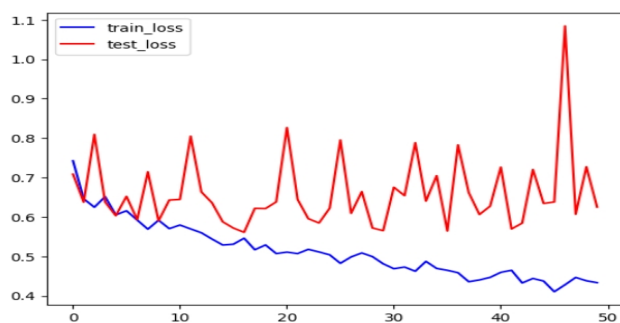
2.4.2 DenseNet

对DenseNet121模型，分别进行从头训练和预处理训练，学习率为0.001，优化算法为Adam，损失函数为交叉熵损失函数。

对从头训练的模型，训练50轮，用时大约5h，结果如下。

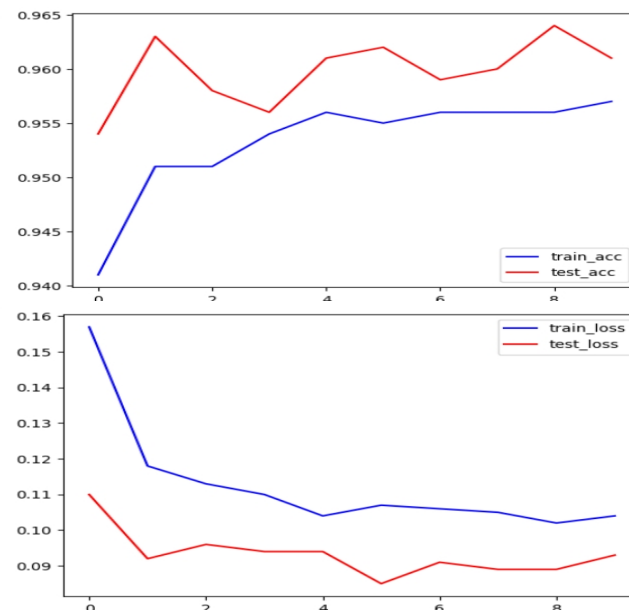
```
Epoch:45, Train_acc:80.1%, Train_loss:0.437, Test_acc:78.2%, Test_loss:0.634, Lr:1.00E-03
train over
Epoch:46, Train_acc:81.6%, Train_loss:0.410, Test_acc:78.3%, Test_loss:0.638, Lr:1.00E-03
train over
Epoch:47, Train_acc:79.8%, Train_loss:0.428, Test_acc:70.7%, Test_loss:1.085, Lr:1.00E-03
train over
Epoch:48, Train_acc:78.9%, Train_loss:0.446, Test_acc:79.5%, Test_loss:0.607, Lr:1.00E-03
train over
Epoch:49, Train_acc:80.0%, Train_loss:0.438, Test_acc:77.3%, Test_loss:0.727, Lr:1.00E-03
train over
Epoch:50, Train_acc:80.3%, Train_loss:0.433, Test_acc:78.8%, Test_loss:0.625, Lr:1.00E-03
Done
```





对于预处理的模型，训练10轮，用时大约0.5h，结果如下。

```
train over
Epoch: 1, Train_acc:94.1%, Train_loss:0.157, Test_acc:95.4%, Test_loss:0.110, Lr:1.00E-03
train over
Epoch: 2, Train_acc:95.1%, Train_loss:0.118, Test_acc:96.3%, Test_loss:0.092, Lr:1.00E-03
train over
Epoch: 3, Train_acc:95.1%, Train_loss:0.113, Test_acc:95.8%, Test_loss:0.096, Lr:1.00E-03
train over
Epoch: 4, Train_acc:95.4%, Train_loss:0.110, Test_acc:95.6%, Test_loss:0.094, Lr:1.00E-03
train over
Epoch: 5, Train_acc:95.6%, Train_loss:0.104, Test_acc:96.1%, Test_loss:0.094, Lr:1.00E-03
train over
Epoch: 6, Train_acc:95.5%, Train_loss:0.107, Test_acc:96.2%, Test_loss:0.085, Lr:1.00E-03
train over
Epoch: 7, Train_acc:95.6%, Train_loss:0.106, Test_acc:95.9%, Test_loss:0.091, Lr:1.00E-03
train over
Epoch: 8, Train_acc:95.6%, Train_loss:0.105, Test_acc:96.0%, Test_loss:0.089, Lr:1.00E-03
train over
Epoch: 9, Train_acc:95.7%, Train_loss:0.102, Test_acc:96.4%, Test_loss:0.089, Lr:1.00E-03
train over
Epoch:10, Train_acc:95.6%, Train_loss:0.104, Test_acc:96.1%, Test_loss:0.093, Lr:1.00E-03
Done
```



整体结果与resnext类似，如果训练轮数能够更多，准确率可以更高。而且预训练每轮所需的时间更短。

2.5 代码附录

代码的zjugit网址: <https://git.zju.edu.cn/3230105182/ResnextandDensenet>

Chapter 3 基于对抗生成网络GAN实现图象生成

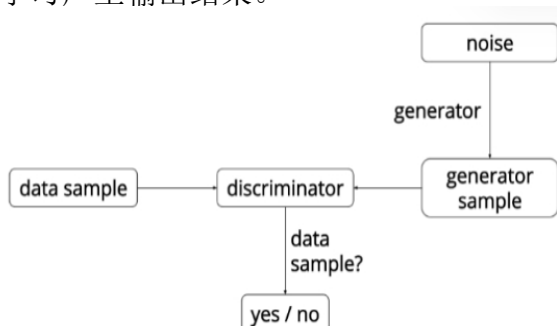
3.1 问题简介

本次实验尝试用对抗生成网络GAN实现生成图象。

3.2 模型介绍

3.2.1 模型原理

生成对抗网络由两个网络组成：生成网络（Generator）和判别网络（Discriminator），G、D两个网络互相博弈学习产生输出结果。



生成器G负责接收一个随机的噪声，通过这个噪声生成图片，将生成的图片送给判别器。判别器D是一个二分类器，用于判断图片是真是假，图片越真，输出的值越接近1，反之接近0。

在训练过程中，G尽量使生成的图片能够欺骗过D，即提高经过D网络判别后的输出概率值。D的目标是提高区分G的假图片和真实图片的能力。这样，G和D就形成了一种博弈过程。

判别器D的训练过程为：随机取样噪声，将噪声输入生成器G得到假图片，其label为0；再随机采样训练数据，即真图片，其label为1。以此作为训练集进行输入，用输出值与原始标签计算损失函数，反向传播更新网络参数。

生成器G的训练过程为：随机取样噪声，输入生成器，输出假图片，将假图片输入判别器，用输出的概率值与目标输出值1计算损失函数，反向传播更新网络参数。

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(x^{(i)}) + \log (1 - D(G(z^{(i)})))]$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)})))$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

两个网络训练时是互相独立的，即一个网络训练时，另一个网络的参数冻结，两者互不干扰。

损失函数用的是交叉熵函数。

$$H(p, q) := - \sum_i p_i \log q_i$$

图象的特征并没有分别用各种标签区分出来，仅仅根据真假标签，G和D可以自行提取出训练数据图象的特征，并更新到网络参数。两者相互博弈训练，最终达到一个动态的平衡（纳什均衡），此时生成器已经提取出了图象的特征，获取了训练数据的分布，判别模型此时也提取出了图象的特征，但是根据这样的“评判标准”，判别器已经判断不出图象的真假（输出为0.5），生成器已经能够以假乱真，双方达到平衡。

3 Chapter 3 基于对抗生成网络GAN实现图象生成

3.2.2 代码实现

```
#定义生成器Generator
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        def block(in_feat, out_feat, normalize=True):
            layers = [nn.Linear(in_feat, out_feat)]
            if normalize:
                layers.append(nn.BatchNorm1d(out_feat, 0.8))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers
        self.model = nn.Sequential(
            *block(opt.latent_dim, 128, normalize=False),
            *block(128, 256),
            *block(256, 512),
            *block(512, 1024),
            nn.Linear(1024, img_area),
            nn.Tanh()
        )
    def forward(self, z):
        imgs = self.model(z)
        imgs = imgs.view(imgs.size(0), *img_shape)
        return imgs
```

此为生成器的网络，每个中间层都由一个linear线性层、一个正则化、一个LeakyRelu激活函数组成。将随机的(100,)的噪声转化为(image_size,)的图象一维展开向量。

```
#定义判别器Discriminator
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(img_area, 512),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )
    def forward(self, img):
        img_flat = img.view(img.size(0), -1)
        validity = self.model(img_flat)
        return validity
```

此为判别器的网络，由三层线性层+LeakyRelu激活函数的组合构成，最后输入sigmoid激活函数进行二分类。

```
#训练判别器
imgs = imgs.view(imgs.size(0), -1)
real_img = Variable(imgs).cuda()
real_label = Variable(torch.ones(imgs.size(0), 1)).cuda()
fake_label = Variable(torch.zeros(imgs.size(0), 1)).cuda()
#计算真的图片的损失
real_out = discriminator(real_img)
loss_real_D = criterion(real_out, real_label)
real_scores = real_out
#计算假的图片的损失
z = Variable(torch.randn(imgs.size(0), opt.latent_dim)).cuda()
fake_img = generator(z).detach()
fake_out = discriminator(fake_img)
loss_fake_D = criterion(fake_out, fake_label)
fake_scores = fake_out
#损失函数和优化
loss_D = loss_real_D + loss_fake_D
optimizer_D.zero_grad()
loss_D.backward()
optimizer_D.step()

#训练生成器
z = Variable(torch.randn(imgs.size(0), opt.latent_dim)).cuda()
fake_img = generator(z)
output = discriminator(fake_img)
#损失函数和优化
loss_G = criterion(output, real_label)
optimizer_G.zero_grad()
loss_G.backward()
optimizer_G.step()

if (i + 1) % 100 == 0:
    print(
        "[Epoch %d/%d] [Batch %d/%d] [D loss: %f] [G loss: %f] [D real: %f] [D fake: %f]"
        % (epoch, opt.n_epochs, i, len(dataloader), loss_D.item(), loss_G.item(), real_scores.data.mean(), fake_scores.data.mean())
    )
#保存训练过程中的图像
batches_done = epoch * len(dataloader) + i
if batches_done % opt.sample_interval == 0:
    save_image(fake_img.data[:25], './images/gan2/%d.png' % batches_done, nrow=5, normalize=True)
```

以上两者的训练函数，与原理基本对应。判别器是获取随机噪声经过生成

器后用输出在经过判别器计算假图片的损失函数；获取随机训练数据经过判别器计算真图片的损失函数，用两个损失函数更新参数；生成器是用随机噪声生成图片经过判别器后的输出计算损失函数，再更新参数。

3.3 数据集介绍

一开始用了上次报告的猫狗数据集，后来发现数据集图片太混乱了，包含大量的环境干扰，导致最后的生成效果不好，如果是纯猫狗面部的图象数据集，效果会很好。

本次用了三个数据集，人脸数据集lfw，动漫头像数据集anime-data以及英雄联盟原画数据集。

lfw有13233张人脸数据集，分别来自5749个人。anime-data有21551张动漫面部头像图片。英雄联盟原画数据集来自其官网，共有8000张不同截图区域的原画，其中有部分重复。

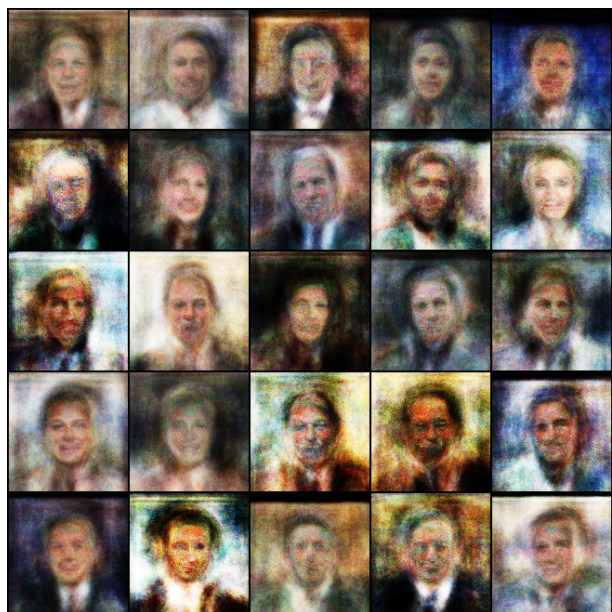
3.4 训练以及结果

参数设置为：学习率0.0002，每次采样数量64，优化器ADAM，其参数b1为0.5，b2为0.999，噪声空间维度100，图象长宽为128，图象通道数为3(彩色图象RGB)，采样间隔500，训练轮数50。全部数据封装在超参数中。

```
#超参数配置
parser = argparse.ArgumentParser()
parser.add_argument("--n_epochs", type=int, default=50, help="训练轮数")
parser.add_argument("--batch_size", type=int, default=64, help="每次采样数量")
parser.add_argument("--lr", type=float, default=0.0002, help="ADAM学习率")
parser.add_argument("--b1", type=float, default=0.5, help="ADAM参数")
parser.add_argument("--b2", type=float, default=0.999, help="ADAM参数")
parser.add_argument("--n_cpu", type=int, default=2)
parser.add_argument("--latent_dim", type=int, default=100, help="噪声空间维度")
parser.add_argument("--img_size", type=int, default=128, help="图象大小")
parser.add_argument("--channels", type=int, default=3, help="图象通道数，彩色图片为三色RGB")
parser.add_argument("--sample_interval", type=int, default=500, help="采样间隔")
opt = parser.parse_args()
print(opt)
```


将训练过程中的图象存在文件夹images中，将训练后的网络参数存在文件夹save中。

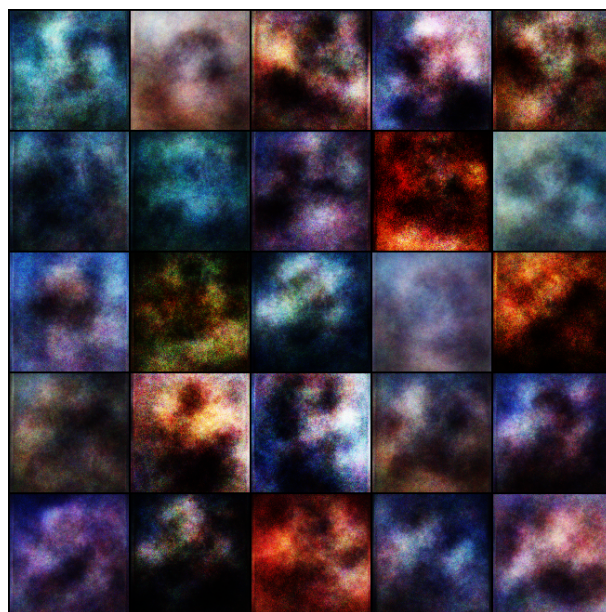
对数据集lfw的训练结果如下图，共训练了50轮。



对anime-data的训练结果如下图，训练50轮。

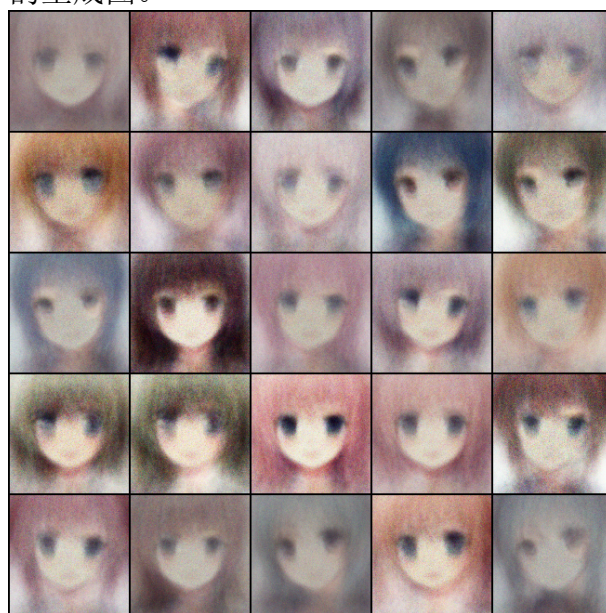


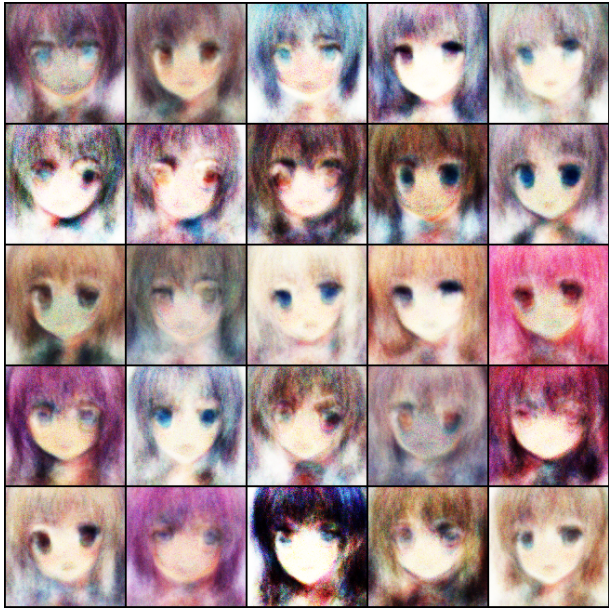
对英雄联盟原画数据集的训练结果如下图，训练50轮。



可见第三个数据集训练结果一般，这也是合理的，原因与猫狗数据集类似，特征不明显或者特征太多，导致没有特别明显的规律，环境比较杂乱。

前两个数据集效果尚可，但是在训练过程中出现了一个问题，就是随着训练轮数的增多，生成图象的饱和度与对比度会逐渐增大。以下分别为采样第1000与采样第7000时的生成图。





确实存在饱和度与对比度逐渐升高的现象，可能是因为在训练过程中，原始数据分布特征逐渐提纯，最后出现过拟合。

3.5 代码附录

代码的zjugit网址: <https://git.zju.edu.cn/3230105182/gan>